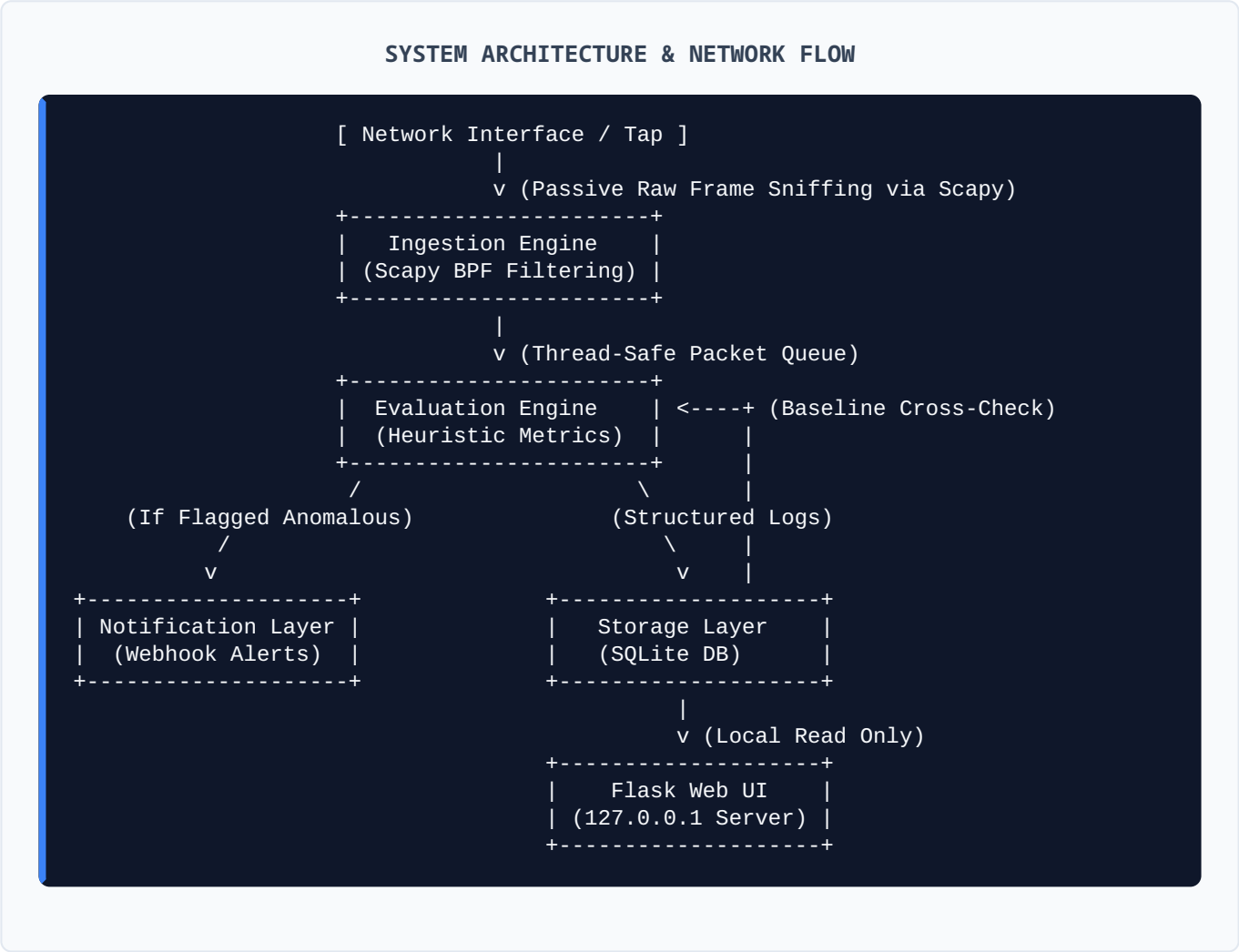


Product Requirement Document (PRD)

Project 7: Localized Traffic Anomaly Detector | Comprehensive Specification for Antigravity Handoff

1. System Architecture & Component Interaction

The Traffic Anomaly Detector is a localized Network Intrusion Detection System (NIDS) built using Python, Scapy, and SQLite. The primary objective is to sniffing network links passively, build historical telemetry profiles, detect deviations, and push immediate alerts via webhooks without modifying the data transmission flight path.



2. Technical Specifications & Database Relational Map

To eliminate excessive disk wear and frame buffer congestion, the storage layer uses index lookup tables structured across three primary operational domains:

SQLite RELATIONSHIP MAP

```
+-----+
|          devices          |
+-----+
| pk | mac_address (TEXT) |
|    | last_known_ip   |
|    | hostname         |
|    | first_seen       |
|    | last_seen        |
+-----+

+-----+
|          alerts          |
+-----+
| pk | id (INTEGER)      |
|    | timestamp       |
|    | alert_type (TEXT) |
|    | description (TEXT) |
|    | severity (TEXT)  |
|    | is_resolved      |
+-----+
```

```
+-----+
|        traffic_logs        |
+-----+
| pk | id (INTEGER)      |
|    | source_ip (TEXT) |
|    | dest_ip (TEXT)  |
|    | dest_port (INTEGER) |
|    | protocol (TEXT) |
|    | packet_count    |
|    | last_active     |
+-----+
```

2.1 Schema Definition SQL Code Block

```
-- Track uniquely identified hardware on the network
CREATE TABLE devices (
    mac_address TEXT PRIMARY KEY,
    last_known_ip TEXT NOT NULL,
    hostname TEXT DEFAULT 'Unknown',
    first_seen TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_seen TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Aggregated traffic logs to prevent table bloat
CREATE TABLE traffic_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    source_ip TEXT NOT NULL,
    dest_ip TEXT NOT NULL,
    dest_port INTEGER NOT NULL,
    protocol TEXT NOT NULL,
    packet_count INTEGER DEFAULT 1,
    last_active TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(source_ip, dest_ip, dest_port, protocol)
);

-- Ledger of triggered alerts
CREATE TABLE alerts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    alert_type TEXT NOT NULL, -- 'NEW_DEVICE', 'UNUSUAL_HOURS', 'SUSPICIOUS_DNS'
    description TEXT NOT NULL,
    severity TEXT CHECK(severity IN ('LOW', 'MEDIUM', 'HIGH')),
    is_resolved INTEGER DEFAULT 0
);
```

3. Explicit Detection Heuristics & Parameters

The evaluation engine processes streaming telemetry down to deterministic thresholds. It isolates baseline characteristics using the following parameters:

Heuristic Trigger	Severity	Condition Parameters
New Device Discovery	MEDIUM	Fires instantly if an unmapped MAC address is discovered in the link-layer frame that is missing from the devices database.
Out-Of-Hours Alert	HIGH	Triggered if a local appliance profile (such as Smart TVs or IoT hardware nodes) passes a payload volume exceeding 50 packets per minute to external WAN gateways between the window of 1:00 AM and 5:00 AM local time.
DNS Tunneling Detection	HIGH	Triggered if a DNS query payload targeted at Port 53 exceeds a total string length of 60 characters or demonstrates a Shannon Entropy score greater than H = 4.5 bits.

4. Step-by-Step Implementation Flow

Antigravity must assemble and configure code packages systematically following this procedural pipeline:



Step 1

Environment Initialization & Configuration

Establish virtual environments. Provision foundational technical assets (scapy, flask, requests). Instantiate the structured SQLite database models using the provided schemas and verify elevated execution permissions on the interface.

Step 2

Ingestion Engine Development

Deploy the Scapy background sniffer logic using Berkeley Packet Filter (BPF) syntax rules to minimize network thread execution overhead. Extract Layer 2, Layer 3, and Layer 4 properties into atomic payloads.

Step 3

Baseline & Threshold Logic Integration

Connect thread-safe packet workers to process data arrays against the state indices. Implement programmatic heuristic triggers checking out-of-hours volumes and tracking high-entropy payloads.

Step 4 Webhook Output Driver Setup

Construct the outbound alert notification driver. Interface analytical alert triggers to remote API destinations (e.g. Discord, Telegram channels) using asynchronous HTTP clients.

Step 5 Flask Interface Presentation Layer

Build the localized browser-accessible monitoring interface. Surface clean inventory metrics and alert history. Securely lock execution bindings to local loopback address 127.0.0.1 to prevent exposing information externally.

5. System Guardrails & Operational Metrics

- **Automated Data Purge Loop:** The engine executes an automated table cleanup script every 24 hours to enforce optimal database sizes:

```
DELETE FROM traffic_logs WHERE last_active < DATETIME('now', '-7 days');
```
- **Memory Constraints:** Total process footprints must be strictly bound below 150MB using dedicated thread workers.
- **Database Locking Defenses:** Read/Write I/O cycles must run separated on an isolated task runner to ensure zero drop occurrences on active capture interfaces.